Testing Automation Tool (TAT)

Enterprise Quality Assurance, Test Lifecycle & Defect Management Platform

Java 21 LTS | Spring Boot 4.1.1 | Spring Cloud Gateway | Angular 22 | Stateless JWT (Dual-Token) | MySQL 8

SYSTEM ARCHITECTURE, SECURITY SPECIFICATION & REST API REFERENCE GUIDE

> ***Target Framework:** Spring Boot 4.1.1 / Java 21 LTS | Angular 22*

> ***Architecture Pattern:** Distributed Microservices with Spring Cloud Gateway & Stateless Dual-Token JWT*

Table of Contents

1. [Executive Overview & Key Capabilities](#1-executive-overview--key-capabilities)
2. [Microservices Architecture & System Topology](#2-microservices-architecture--system-topology)
3. [Service Catalog & Network Port Matrix](#3-service-catalog--network-port-matrix)
4. [Security, Authentication & RBAC Architecture](#4-security-authentication--rbac-architecture)
5. [Domain Model & Entity-Relationship Architecture](#5-domain-model--entity-relationship-architecture)
6. [Core Business Workflows & Modules Deep-Dive](#6-core-business-workflows--modules-deep-dive)
   - [6.1 User Onboarding & Approval Lifecycle](#61-user-onboarding--approval-lifecycle)
   - [6.2 Project & Agile Feature Management](#62-project--agile-feature-management)
   - [6.3 Test Case Authoring & Bulk Excel Engine](#63-test-case-authoring--bulk-excel-engine)
   - [6.4 Multi-Tier Test Execution Logging](#64-multi-tier-test-execution-logging)
   - [6.5 Defect State Machine & Automated ID Engine](#65-defect-state-machine--automated-id-engine)
   - [6.6 Defect Collaboration & Media Attachment Subsystem](#66-defect-collaboration--media-attachment-subsystem)
   - [6.7 Outlook SMTP Notification Service](#67-outlook-smtp-notification-service)
   - [6.8 Streaming QA Summary & Export Microservice](#68-streaming-qa-summary--export-microservice)
   - [6.9 Manager Quality Health Score Algorithm](#69-manager-quality-health-score-algorithm)
7. [Frontend Architecture & Single Page Application (SPA)](#7-frontend-architecture--single-page-application-spa)
8. [Comprehensive REST API Reference](#8-comprehensive-rest-api-reference)
9. [Configuration & Environment Variables](#9-configuration--environment-variables)
10. [Local Development, Build & Startup Sequence](#10-local-development-build--startup-sequence)

---

1. Executive Overview & Key Capabilities

The **Testing Automation Tool (TAT)** is an enterprise Quality Assurance (QA), Test Lifecycle Management, and Defect Tracking ecosystem. It consolidates disparate QA processes-test case authoring, bulk spreadsheet operations, test run validation, automated bug formatting, audit logging, defect team discussions, email notifications, and executive quality scoring-into a unified, distributed platform.

Key Capabilities

- **Role-Based Access Control (RBAC)**: Enforced segregation of duties across `ADMIN`, `MANAGER`, `TESTER`, and `DEVELOPER`.
- **Manager Verification Pipeline**: Self-registered users are quarantined in an inactive state until vetted, approved, and assigned an operational role by a Manager.
- **Flexible Test Suite Management**: Custom user-defined attributes stored via JSON type mapping (`@JdbcTypeCode(SqlTypes.JSON)`).
- **High-Performance Spreadsheet Engine**: Dynamic template generation and bulk `.xlsx` test case parsing using Apache POI (SXSSF streaming for large datasets).
- **Multi-Tier Execution Tracking**: Test executions capture UI validations, database query validations, preconditions, and test data alongside manual/automated run statuses.

- **Automated Defect Formatting & State Machine**: Standardized bug identifiers (e.g. `BUG-AUTH_MODULE-001`), immutable audit trail logs in `testing_bug_history`, and root-cause analysis (RCA) reporting.
- **Defect Collaboration & Attachment Hub**: Threaded commenting and multi-part attachment uploading with automated `.zip` bundle downloads.
- **Enterprise SMTP Dispatcher**: Dedicated microservice handling asynchronous HTML notifications via Microsoft Outlook / Office 365 SMTP relay with delivery tracking.
- **Proprietary Quality Health Scoring**: Real-time evaluation of software stability based on test pass rates, execution density, and weighted defect severity penalties.

2. Microservices Architecture & System Topology

TAT is architected as a distributed microservice ecosystem behind a centralized Edge API Gateway.

```
graph TD
    subgraph ClientLayer ["Client Presentation Layer"]
        AngularApp["Angular 22 SPA (Port 4200)"]
        LegacyUIs["Static HTML Consoles (manager-dashboard, bug-chat)"]
    end

    subgraph EdgeLayer ["API Gateway & Routing (Port 8082)"]
        Gateway["Spring Cloud Gateway WebFlux"]
        CorsEngine["Global CORS & Header Deduplicator"]
    end

    subgraph CoreMicroservices ["Backend Microservices Ecosystem"]
        AuthSvc["auth-service (Port 8081)<br/>* Identity & Registration<br/>* Dual-Token JWT (A
        MainSvc["main-service (Port 8090)<br/>* Projects & Features<br/>* Test Cases & Executio
        SummarySvc["summary-export-service (Port 8092)<br/>* SXSSF Streaming Excel Export<br/>*
        NotifSvc["notification-service (Port 8091)<br/>* Office 365 / Outlook SMTP Relay<br/>*
    end

    subgraph SharedModules ["Shared Libraries"]
        CommonLib["common module<br/>* JwtUtils & CookieUtils<br/>* SpaCsrfTokenRequestHandler<
    end

    subgraph DataStorage ["Data Tier (MySQL)"]
        AppDB[("Interns_db (Core Database)<br/>* Users, Tokens, Projects<br/>* Features, Tests,
        NotifDB[("notification_service_db<br/>* Notification Tracking Table")]
    end

    ClientLayer -->|HTTP Requests with Cookies & CSRF| Gateway
    Gateway --> CorsEngine
    CorsEngine -->|Route /auth/**| AuthSvc
    CorsEngine -->|Route /manager/**, /project/**, /feature/**, /testcases/**, /bugs/**, /admin
    CorsEngine -->|Route /api/v1/notifications/**| NotifSvc

    AuthSvc -.->|Trigger OTP Emails| NotifSvc
    MainSvc -.->|Trigger Event Notifications| NotifSvc
```

3. Service Catalog & Network Port Matrix

Service / Component	Sub-directory	Runtime / Tech Stack	Default Port	Primary Responsibility
Angular Frontend	`Testing_Automation_tool_frontend/testing-automation-frontend`	Node.js 18+, Angular 22, TypeScript	`4200`	Single Page Application providing interactive portals for Admin, Manager, Tester, and Developer workflows.
API Gateway	`TestingAutomationTool/api-gateway`	Spring Boot 4.1.1, Spring Cloud Gateway WebFlux	`8082`	Public reverse proxy, global CORS management, cookie passthrough, and downstream route dispatching.
Auth Service	`TestingAutomationTool/auth-service`	Spring Boot 4.1.1, Spring Security, JJWT	`8081`	User registration, authentication, dual-token generation, refresh token persistence, and 6-digit email OTP password recovery.

Main Service	`TestingAutomationTool/main-service`	Spring Boot 4.1.1, Spring Data JPA, Hibernate 6	`8090`	Core business engine: Project, Feature, TestCase, Execution, Bug, Comment, Attachment, Admin, and Executive Dashboard.
Notification Service	`notification-service`	Spring Boot, Spring Mail, Jakarta Mail	`8091`	Standalone email notification microservice interfacing with Microsoft Outlook/Office 365 SMTP with delivery persistence.
Summary Export Service	`summary-export-service`	Spring Boot 4.1.1, Apache POI SXSSF	`8092`	High-throughput, memory-efficient Excel report generation and dynamic test case upload template builder.
Common Shared Library	`TestingAutomationTool/common`	Java 21 Library	N/A	Shared security filters, JWT utilities, cookie helpers, CSRF handlers, entity models, and DTO contracts.

4. Security, Authentication & RBAC Architecture

The platform enforces a zero-trust, defense-in-depth security model across all tiers:

```
sequenceDiagram
    autonumber
    actor User as Client Browser
    participant Gateway as API Gateway (:8082)
    participant Auth as Auth Service (:8081)
    participant Main as Main Service (:8090)
    participant DB as MySQL (Interns_db)

    Note over User,Auth: Authentication & Token Issuance
    User->>Gateway: POST /auth/login (username, password)
    Gateway->>Auth: Forward to :8081
    Auth->>DB: Verify credentials & active account status
    Auth->>DB: Store persistent refresh token in testing_refresh_tokens
    Auth-->>Gateway: 200 OK + Set-Cookie: accessToken (15m HttpOnly)<br/>+ Set-Cookie: refreshT
    Gateway-->>User: Pass Set-Cookie headers back to browser

    Note over User,Main: Protected Business Request
    User->>Gateway: POST /testcases (with Cookies + X-XSRF-TOKEN header)
    Gateway->>Main: Forward request to :8090 with headers & cookies
    Main->>Main: CsrfCookieFilter & SpaCsrfTokenRequestHandler validate CSRF
    Main->>Main: JwtAuthenticationFilter validates accessToken signature & claims
    Main->>Main: Check @PreAuthorize("hasAnyRole('MANAGER', 'TESTER', 'ADMIN')")
    Main->>DB: Execute business logic
    Main-->>Gateway: 201 Created
    Gateway-->>User: 201 Created
```

4.1 Dual-Token Lifecycle

- **Access Token (`accessToken`)**:
- Validity: **15 minutes** (`900,000 ms`).
- Carries claims: `userId`, `username`, `email`, `role`, `fullName`.
- Stored in an **HttpOnly**, `SameSite=Lax` browser cookie.
- **Refresh Token (`refreshToken`)**:
- Validity: **24 hours** (`86,400,000 ms`).
- Persisted in MySQL table `testing_refresh_tokens`.
- Exchanged via `POST /auth/refresh` without prompting the user for credentials.
- **10-Minute Session Warning**: If refreshed within the last 10 minutes of its 24-hour lifetime, the API returns a warning notification (`sessionExpiryWarning`).
- **Strict Session Expiration**: Once expired or revoked, calls return `HTTP 401 Unauthorized` with `"Refresh token expired. Please log in again."`.

4.2 In-Memory OTP Password Recovery

1. `POST /auth/forgot-password`: Generates a cryptographically random 6-digit OTP stored in thread-safe memory with a **10-minute TTL**. Dispatches an HTML email via `notification-service`.
2. `POST /auth/verify-otp`: Validates code correctness without changing state.
3. `POST /auth/reset-password`: Validates OTP, updates password with BCrypt, invalidates all prior refresh tokens, and returns fresh Access + Refresh tokens for an instant authenticated session.

4.3 Double-Submit CSRF Defense

- Uses Spring Security 6's deferred CSRF model.
- `CsrfCookieFilter` forces generation of the `XSRF-TOKEN` cookie.
- `SpaCsrfTokenRequestHandler` processes the incoming `X-XSRF-TOKEN` header on non-safe HTTP methods (`POST`, `PUT`, `PATCH`, `DELETE`).

4.4 Role-Based Access Control (RBAC) Matrix

Functional Area	Path Pattern	Required Role(s)	Description
Public Auth	`/auth/register`, `/auth/login`, `/auth/refresh`, `/auth/forgot-password`, `/auth/reset-password`	Public	Unauthenticated user operations
Session & Profile	`/auth/profile`, `/auth/change-password`, `/auth/logout`	Authenticated	Any active logged-in user
Manager Approvals	`/manager/users/pending`, `/manager/userConfirmation`, `/manager/userRejection/*`	`MANAGER`, `ADMIN`	Vetting and activating newly registered user accounts
Manager Analytics	`/manager/dashboard/**`	`MANAGER`, `ADMIN`	Real-time QA metrics and health scoring
User Administration	`/admin/**`	`ADMIN`	CRUD on managers, user role adjustments, soft/hard deletion
Project Management	`/project`, `/project/{id}`, `/project/search`, `/project/attachment/*`	`MANAGER`, `TESTER`, `ADMIN`	Manage testing projects and file documentation
Project Deletion	`PATCH /project/access/{id}`, `DELETE /project/**`	`MANAGER`	Soft-disable, soft-delete, or permanent purge
Feature Management	`/feature/**`	`MANAGER`, `TESTER`, `ADMIN`	Sprints, versions, and attachments within projects
Test Case Suite	`/testcases/**`	`MANAGER`, `TESTER`, `ADMIN`	Test case authoring, execution logging, Excel upload
Defect Tracking	`/bugs/**`	`MANAGER`, `TESTER`, `ADMIN`	Bug lifecycle, status transitions, attachments
Defect Discussion	`/comments/**`, `/bugs/{bugId}/comments/**`	`MANAGER`, `TESTER`, `ADMIN`	Threaded discussions on defects
In-App Alerts	`/in-app-notifications/**`	Authenticated	User notification inbox and mark-as-read updates
Summary Reports	`/export/summary/**`, `/export/template/**`	`MANAGER`, `TESTER`, `ADMIN`	Project/feature Excel export and upload templates

---

5. Domain Model & Entity-Relationship Architecture

```
erDiagram
    ROLE ||--o{ USER : "assigned to"
    USER ||--o{ REFRESH_TOKEN : "owns"
    USER ||--o{ PROJECT : "creates"
    USER ||--o{ FEATURE : "creates"
    USER ||--o{ TEST_CASE : "authors"
    USER ||--o{ TESTING_EXECUTION : "executes"
    USER ||--o{ BUG : "reports / assigned to"
    USER ||--o{ COMMENT : "authors"
    USER ||--o{ IN_APP_NOTIFICATION : "receives"

    PROJECT ||--o{ FEATURE : "contains"
    PROJECT ||--o{ ATTACHMENT : "has documents"

    FEATURE ||--o{ TEST_CASE : "groups"
    FEATURE ||--o{ BUG : "encounters"
    FEATURE ||--o{ ATTACHMENT : "has documents"

    TEST_CASE ||--|| TESTING_EXECUTION : "1-to-1 execution details"
    TEST_CASE ||--o{ BUG : "generates"
```

```
    BUG ||--o{ BUG_HISTORY : "tracks state transitions"
    BUG ||--o{ COMMENT : "discussion thread"
    BUG ||--o{ ATTACHMENT : "screenshot / logs evidence"
```

Core Entity Schema Definitions

- **`User` (`testing_users`)**: ID, username, email, password (BCrypt), full name, designation, skills, role reference, active status (`isActive`), soft-delete flag (`isDeleted`).
- **`RefreshToken` (`testing_refresh_tokens`)**: ID, user reference, token hash, expiry timestamp, revocation flag (`isRevoked`).
- **`Project` (`testing_projects`)**: ID, name, region, status (`ACTIVE`, `INACTIVE`), soft-delete flag, optimistic lock version.
- **`Feature` (`testing_features`)**: ID, project reference, name, sprint number, status (`ACTIVE`, `IN_PROGRESS`, `COMPLETED`, `ON_HOLD`, `DEPRECATED`), feature version.
- **`TestCase` (`testing_testcases`)**: ID, feature reference, formatted ID (`testcaseFormatId`), title, test type (`SMOKE`, `REGRESSION`, `FUNCTIONAL`, `SANITY`, `E2E`), test priority, status, dynamic JSON metadata (`dynamicFields`).
- **`TestingExecution` (`testing_executions`)**: ID, testcase reference, execution status (`PASS`, `FAIL`, `NO_RUN`, `BLOCKED`, `DESCOPE`), automation feasibility (`YES`, `NO`, `PLANNED`), UI validations, DB validations, preconditions, test data, executed-at timestamp.
- **`Bug` (`testing_bugs`)**: ID, formatted ID (`bugFormatId`), testcase reference, feature reference, title, severity (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`), priority, category (`PRE_PRODUCTION`, `PRODUCTION`, `REGRESSION`, `SECURITY`, `PERFORMANCE`, `UI_UX`), status (`OPEN`, `IN_PROGRESS`, `RESOLVED`, `FIXED`, `REOPENED`, `CLOSED`), reporter, assignee, RCA comments.
- **`BugHistory` (`testing_bug_history`)**: ID, bug reference, previous status, updated status, previous assignee, new assignee, change comments, modified timestamp, changed-by user.
- **`Comment` (`testing_comments`)**: ID, bug reference, user reference, message body, created/updated timestamps.
- **`Attachment` (`testing_attachments`)**: ID, entity type (`PROJECT`, `FEATURE`, `BUG`), entity ID, original file name, stored file path, MIME type, file size.

6. Core Business Workflows & Modules Deep-Dive

6.1 User Onboarding & Approval Lifecycle

1. **Self-Registration**: Candidate users fill out `/auth/register`. The account is created with `isActive = false`.
2. **Review Queue**: Managers view pending registrations at `GET /manager/users/pending`.
3. **Approval (`PUT /manager/userConfirmation`)**: Manager approves the user, assigning their official role (`TESTER`, `DEVELOPER`, `MANAGER`). An automated welcome email is dispatched via `notification-service`.
4. **Rejection (`DELETE /manager/userRejection/{userId}`)**: Purges the registration request and sends a polite rejection notice.

6.2 Project & Agile Feature Management

- Projects represent overarching software initiatives partitioned by geographical deployment regions.
- Features represent Agile sprints or epics.
- Document repository: Files (specifications, architecture diagrams) can be attached to projects or features and downloaded individually or bundled into ZIP archives.

6.3 Test Case Authoring & Bulk Excel Engine

- **Manual Mode**: Test cases are created via `POST /testcases` with custom JSON dynamic attributes.
- **Bulk Excel Import**: Testers download a formatted template (`GET /testcases/template/{projectId}/{featureId}`), populate it, and upload via `POST /testcases/upload`.
- Sheet Name: `TCFormat`
- Columns: Test Case ID, Title, Test Type, Test Execution, Test Validation, Automation Priority, Preconditions, Test Data, Execution Steps, UI Validations, DB Validations, Automation Feasibility, Actual Status, Comments.
- The POI parser validates rows, converts statuses, and generates corresponding `TestCase` and `TestingExecution` entities in a single atomic transaction.

6.4 Multi-Tier Test Execution Logging

Validations are segregated into UI and database tiers:

- **UI Validations**: Visual and DOM assertion logs.
- **DB Validations**: SQL query checks verifying data persistence and referential integrity.
- **Statuses**: `PASS`, `FAIL`, `NO_RUN`, `BLOCKED`, `DESCOPE`.

6.5 Defect State Machine & Automated ID Engine

- **Bug ID Generation**: Standardized pattern `BUG-<FEATURE_NAME>-<SEQUENCE_NUMBER>` (e.g. `BUG-AUTHENTICATION-001`).
- **Defect State Transitions**:

```
stateDiagram-v2
    [*] --> OPEN: Bug Logged
    OPEN --> IN_PROGRESS: Developer Investigates
    IN_PROGRESS --> RESOLVED: Fix Submitted
    IN_PROGRESS --> BLOCKED: Blocked by Dependency
    BLOCKED --> IN_PROGRESS: Unblocked
    RESOLVED --> FIXED: Developer/Build Verification
    FIXED --> CLOSED: Tester Verifies Pass
    FIXED --> REOPENED: Tester Finds Failure
    REOPENED --> IN_PROGRESS: Developer Rework
    CLOSED --> [*]
```

- Every state or assignee change creates an audit record in `testing_bug_history` and dispatches email and in-app notifications.

6.6 Defect Collaboration & Media Attachment Subsystem

- **Threaded Discussions**: In-context comments between testers and developers.
- **File Uploads**: Screenshots, log files, stack traces, or screen recordings up to 20MB per file.
- **ZIP Packaging**: Download all evidence files for a bug in a single compressed archive.

6.7 Outlook SMTP Notification Service

- Independent microservice (`:8091`) accepting requests from `main-service` and `auth-service`.
- Connects to Microsoft Office 365 / Outlook SMTP via STARTTLS on port 587.
- Enforces attachment security (prohibits `.exe`, `.bat`, `.sh` scripts).
- Persists notification tracking status (`PENDING`, `SUBMITTED`, `FAILED`) in `notification_service_db`.
- Does not log or persist sensitive message bodies to protect user confidentiality.

6.8 Streaming QA Summary & Export Microservice

- High-efficiency export service (`:8092`) utilizing Apache POI's SXSSF streaming workbook.
- Streams comprehensive multi-tab reports for entire projects or specific features:
- Tab 1: Executive Metrics & Health Score summary.
- Tab 2: Test Case catalog & execution details.
- Tab 3: Defect inventory with severities and RCA comments.

6.9 Manager Quality Health Score Algorithm

Calculated in `DashboardServiceImpl`:

$$\text{Pass Rate (\%)} = \frac{\text{Passed Test Cases}}{\text{Passed} + \text{Failed}} \times 100$$

$$\text{Base Score} = (\text{Pass Rate} \times 0.75) + 25.0$$

$$\text{Final Health Score} = \text{clamp}\Big(\text{Base Score} - (\text{Critical Defects} \times 20.0) - (\text{High Defects} \times 8.0), \, 0.0, \, 100.0 \Big)$$

Rating Classifications

- **EXCELLENT** ($\ge 85\%$ score, $\ge 85\%$ pass rate, 0 critical defects)
- **GOOD** ($\ge 70\%$ score)
- **NEEDS_ATTENTION** ($\ge 50\%$ score)
- **CRITICAL** ($< 50\%$ score OR any open Critical defect)

---

7. Frontend Architecture & Single Page Application (SPA)

The user interface is built on **Angular 22** using standalone components, Angular routing, and reactive services.

Routing Table & Access Controls

```
/login                    -> Login Component (Public)
/register                 -> Register Component (Public)
/forgot-password          -> Forgot Password (Public)
/verify-otp               -> Verify 6-digit OTP (Public)
/reset-password           -> Set New Password (Public)
/profile                  -> User Profile (authGuard)
/tester-workspace         -> Tester Workspace (authGuard)
/developer-workspace      -> Developer Workspace (authGuard)
/manager-dashboard        -> Manager Dashboard (authGuard, managerGuard)
/admin-workspace          -> Admin Workspace (authGuard)
```

Key Frontend Core Services & Interceptors

- **`authService.ts`**: Handles authentication, user profile state, session timeout monitoring, and cookie credentials.
- **`auth.interceptor.ts`**: Attaches `withCredentials: true` to all outgoing HTTP requests, extracts the `XSRF-TOKEN` cookie, and injects the `X-XSRF-TOKEN` header. Automatically redirects to login on `401 Unauthorized`.
- **`proxy.conf.json`**: Proxies `/api/*` to `http://localhost:8082` during local development (`ng serve`), stripping the `/api` prefix.

---

8. Comprehensive REST API Reference

All requests passing through the API Gateway use base URL: `http://localhost:8082` (or direct service ports for internal communication).

8.1 Authentication & Profile (`/auth/**` -> Port 8081)

- `POST /auth/register` - Public user registration.
- `POST /auth/login` - Authenticate and receive `accessToken` + `refreshToken` HttpOnly cookies.
- `POST /auth/refresh` - Issue fresh access token from valid refresh token.
- `POST /auth/forgot-password` - Request 6-digit email OTP.
- `POST /auth/verify-otp` - Verify OTP validity.
- `POST /auth/reset-password` - Reset password using OTP with auto-login.
- `POST /auth/logout` - Invalidate refresh token and clear cookies.
- `GET /auth/profile` - Get authenticated user details.
- `POST /auth/change-password` - Update user password.

8.2 Project Management (`/project/**` -> Port 8090)

- `POST /project` - Create project.
- `GET /project` - List all active projects.
- `GET /project/{id}` - Get project by ID.
- `GET /project/search?keyword=&status=` - Filter projects.
- `PUT /project/{id}` - Update project.
- `PATCH /project/{id}` - Partial update.
- `PATCH /project/access/{id}` - Toggle active status (`MANAGER`).
- `DELETE /project/delete/{id}` - Soft delete project (`MANAGER`).
- `DELETE /project/{id}` - Permanent delete (`MANAGER`).
- `POST /project/attachment/{projectId}` - Upload documentation attachment.
- `GET /project/projects/{projectId}/attachments/download` - Download all attachments as ZIP.

8.3 Feature Management (`/feature/**` -> Port 8090)

- `POST /feature` - Create feature under a project.
- `GET /feature/{id}` - Get feature details.
- `GET /feature/project/{projectId}` - List all features in a project.
- `GET /feature/search?keyword=&status=&page=&size=` - Paginated feature search.
- `PUT /feature/{id}` - Update feature.
- `PATCH /feature/{id}` - Partial update.
- `DELETE /feature/{id}` - Soft delete feature.
- `POST /feature/attachment/{featureId}` - Upload feature document.
- `GET /feature/features/{featureId}/attachments/download` - Download feature attachments ZIP.

8.4 Test Case Suite (`/testcases/**` -> Port 8090)

- `POST /testcases` - Create test case with execution criteria.
- `POST /testcases/upload` - Bulk import via `.xlsx` spreadsheet upload.
- `GET /testcases` - Paginated list of test cases.

- `GET /testcases/{id}` - Retrieve test case by ID.
- `GET /testcases/search` - Filter by feature, status, priority, type.
- `PUT /testcases/{id}` - Update test case and execution details.
- `PATCH /testcases/{id}` - Partial status/priority update.
- `DELETE /testcases/soft/{id}` - Soft delete.
- `DELETE /testcases/{id}` - Hard delete.

8.5 Defect & Bug Tracking (`/bugs/**` -> Port 8090)

- `POST /bugs` - Log a new bug (auto-assigns `BUG-<FEATURE>-XXX`).
- `GET /bugs/{id}` - Get bug details.
- `GET /bugs?page=&size=&sort=` - Paginated list of defects.
- `PUT /bugs/{id}` - Full update of defect.
- `PATCH /bugs/{id}` - Update status or assignee (triggers email & audit history).
- `GET /bugs/search` - Search bugs by severity, category, priority, status.
- `PATCH /bugs/delete/{id}` - Soft delete defect.
- `DELETE /bugs/{id}` - Hard delete defect.
- `POST /bugs/{bugId}/attachments` - Upload bug screenshot or log.
- `GET /bugs/{bugId}/attachments` - List attachments for bug.
- `GET /bugs/attachments/{attachmentId}/download` - Download specific attachment.

8.6 Defect Comments (`/bugs/{bugId}/comments` & `/comments/**` -> Port 8090)

- `POST /bugs/{bugId}/comments` - Add comment to defect.
- `GET /bugs/{bugId}/comments` - List all comments for defect.
- `GET /bugs/{bugId}/comments/page` - Paginated comments list.
- `PUT /comments/{commentId}` - Edit comment.
- `DELETE /comments/{commentId}` - Delete comment.

8.7 Manager Dashboard & Approvals (`/manager/**` -> Port 8090)

- `GET /manager/dashboard/overview` - High-level KPIs, quality health score, top failing features.
- `GET /manager/dashboard/project/{projectId}` - Project-specific test & defect metrics.
- `GET /manager/dashboard/feature/{featureId}` - Feature-specific metrics.
- `GET /manager/users/active` - List active users.
- `GET /manager/users/pending` - List pending sign-up requests.
- `PUT /manager/userConfirmation` - Approve user and set role.
- `DELETE /manager/userRejection/{userId}` - Reject pending signup.

8.8 User Administration (`/admin/**` -> Port 8090)

- `POST /admin/managers` - Create manager account.
- `GET /admin/users` - List all system users.
- `GET /admin/users/{userId}` - Retrieve user profile.
- `GET /admin/users/role/{role}` - List users by role.
- `PUT /admin/users/{userId}` - Update user profile and role.
- `DELETE /admin/users/{userId}` - Soft delete user.
- `DELETE /admin/users/{userId}/permanent` - Permanent user purge.

8.9 Summary & Export Microservice (`/export/**` -> Port 8092)

- `GET /export/summary/project/{projectId}` - Download full project QA Excel summary.
- `GET /export/summary/feature/{featureId}` - Download feature QA Excel summary.
- `GET /export/template/{projectId}/{featureId}` - Download `.xlsx` test case template.

8.10 Notification Microservice (`/api/v1/notifications/**` -> Port 8091)

- `POST /api/v1/notifications/email` - Submit an email for dispatch.
- `GET /api/v1/notifications/{notificationId}` - Check email delivery status.
- `GET /api/v1/notifications` - Paginated history of dispatched emails.

9. Configuration & Environment Variables

Variable	Target Service	Default	Purpose

`DB_URL`	Auth, Main, Summary	`jdbc:mysql://192.168.11.101:3306/interns_db`	JDBC connection string for core application database.
`DB_USERNAME`	Auth, Main, Summary	*(Configured)*	MySQL database username.
`DB_PASSWORD`	Auth, Main, Summary	*(Configured)*	MySQL database password.
`JWT_SECRET`	Auth, Main, Summary	`404E635266556A586E32...`	256-bit secret key for HMAC-SHA256 signature validation.
`JWT_ACCESS_TOKEN_EXPIRATION_MS`	Auth, Main	`900000` (15m)	Lifespan of access token.
`JWT_REFRESH_TOKEN_EXPIRATION_MS`	Auth, Main	`86400000` (24h)	Lifespan of refresh token.
`NOTIFICATION_SERVICE_URL`	Auth, Main, Gateway	`http://localhost:8091`	Base URL for the Notification Microservice.
`AUTH_SERVICE_URL`	Gateway	`http://localhost:8081`	Downstream route for `/auth/**`.
`MAIN_SERVICE_URL`	Gateway	`http://localhost:8090`	Downstream route for business operations.
`OUTLOOK_SMTP_HOST`	Notification	`smtp.office365.com`	SMTP host for email dispatch.
`OUTLOOK_SMTP_PORT`	Notification	`587`	STARTTLS SMTP port.
`OUTLOOK_SMTP_USERNAME`	Notification	*(Configured)*	Office 365 / Outlook mailbox address.
`OUTLOOK_SMTP_PASSWORD`	Notification	*(Configured)*	Mailbox app password.
`OUTLOOK_DEFAULT_FROM`	Notification	`notifications@example.com`	Default sender email address.

---

10. Local Development, Build & Startup Sequence

Prerequisites

- **Java SE Development Kit**: Version 21 LTS
- **Node.js**: Version 18.x or 20.x
- **Angular CLI**: Version 22.x
- **Apache Maven**: Version 3.9+ (or use `./mvnw`)
- **MySQL Server**: 8.0+

Recommended Startup Sequence

1. Databases

Ensure both MySQL databases are running and accessible:

- `Interns_db` (Core application schema)
- `notification_service_db` (Notification tracking schema, initialize with `notification-service/schema.sql`)

2. Notification Microservice (Port 8091)

```
cd D:\Testing_Automation_Tool\notification-service
./mvnw clean spring-boot:run
```

3. Core TestingAutomationTool Modules

From directory `D:\Testing_Automation_Tool\TestingAutomationTool`:

```
cd D:\Testing_Automation_Tool\TestingAutomationTool
# Build shared library and all submodules
./mvnw clean package -DskipTests
```

Start the individual submodules:

```
# Terminal 1: Auth Service (Port 8081)
cd auth-service
../mvnw spring-boot:run

# Terminal 2: Main Business Service (Port 8090)
cd main-service
../mvnw spring-boot:run

# Terminal 3: API Gateway (Port 8082)
cd api-gateway
../mvnw spring-boot:run
```

4. Summary & Export Microservice (Port 8092)

```
cd D:\Testing_Automation_Tool\summary-export-service
./mvnw clean spring-boot:run
```

5. Angular Frontend (Port 4200)

```
cd D:\Testing_Automation_Tool\Testing_Automation_tool_frontend\testing-automation-frontend
npm install
npm start   # Runs 'ng serve --proxy-config proxy.conf.json'
```

Open your browser to `http://localhost:4200` to access the Testing Automation Tool.